

Custom WhatsApp CRM Platform

PROPRIETARY SOFTWARE REQUIREMENT SPECIFICATIONS & ENGINEERING COST BLUEPRINT

This architectural document outlines the engineering specification, system requirements, direct API integration mappings, database design, and operational cost blueprints required to deploy a proprietary, multi-agent WhatsApp CRM and high-volume marketing engine. By communicating directly with the official Meta Cloud API, this platform operates autonomously without dependencies on third-party aggregators or subscription middleware layers.

1. System Architecture & Component Mapping

To support asynchronous operational pipelines (such as continuous conversational handling mixed with massive outbound marketing broadcasts), the application framework splits duties across separate specialized layers. This keeps high-volume traffic from locking up individual UI interactions.

- **Frontend Framework:** Single Page Application (SPA) built via React.js optimized with Tailwind CSS for dynamic layout refreshing.
- **Runtime Server Backend:** Node.js or Python runtime parsing requests within lightweight asynchronous routers.
- **Primary Storage Engine:** PostgreSQL relational database configuring ACID-compliant transactions, contact tagging, and message logs.
- **Job Broker / Memory Cache:** Redis caching pipeline organizing strict token-bucket transaction pacing and WebSocket data delivery.
- **Real-Time Broadcast Server:** WebSockets handling immediate live messaging states across simultaneous internal agent sessions.

2. Operational Database Schema

The core infrastructure requires clear transactional tracking for contacts, support team access, and message deliverability telemetry. The relational model maps out these paths to maintain precise structural data limits.

```

-- 1. CONTACTS: Target segment records and properties
CREATE TABLE contacts (
    id SERIAL PRIMARY KEY,
    phone_number VARCHAR(20) UNIQUE NOT NULL, -- Format: E.164 (e.g., 919876543210)
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    tags TEXT[], -- Example: ['warm_lead', 'campaign_jun']
    custom_properties JSONB, -- Variable metadata fields
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. AGENTS: Internal support or sales accounts
CREATE TABLE agents (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role VARCHAR(20) DEFAULT 'agent', -- 'admin', 'manager', 'agent'
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. MESSAGES: Interaction audit ledger
CREATE TABLE messages (
    id SERIAL PRIMARY KEY,
    whatsapp_message_id VARCHAR(255) UNIQUE, -- Direct string key from Meta API
    contact_id INT REFERENCES contacts(id) ON DELETE CASCADE,
    agent_id INT REFERENCES agents(id) ON DELETE SET NULL,
    direction VARCHAR(10) NOT NULL, -- 'inbound' or 'outbound'
    message_type VARCHAR(20) NOT NULL, -- 'text', 'image', 'document', 'template'
    body TEXT, -- Extracted payload content
    media_url TEXT, -- Local or cloud link to assets
    status VARCHAR(20) DEFAULT 'queued', -- 'queued', 'sent', 'delivered', 'read', 'fail
    error_message TEXT, -- Detailed error context if failed
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Optimization Performance Indexes
CREATE INDEX idx_contacts_phone ON contacts(phone_number);
CREATE INDEX idx_messages_contact_id ON messages(contact_id);
CREATE INDEX idx_messages_status ON messages(status);

```

3. Direct Meta Cloud API Requirements

All programmatic interaction uses the safe HTTPS endpoints provided by Meta. The core gateway setup integrates four explicit structural categories:

A. Conversational Outbound Processing

Sends template alerts or standard responses directly. Standard responses are permitted when a conversation window has been open for less than 24 hours.

- **Endpoint:** POST `https://graph.facebook.com/v18.0/{phone_number_id}/messages`
- **Headers:** Authorization: Bearer Access_Token, Content-Type: application/json

B. Account Message Template Ingestion

Pulls verified templates, verifies rejection alerts, or submits structural copywriting drafts to the automated validation dashboard.

- **Endpoint:** GET/POST `https://graph.facebook.com/v18.0/{whatsapp_business_account_id}/whats_app_templates`

C. Binary Asset Upload Gateway

Converts uploaded documents, bills, or images into persistent Meta media references used in outbound message objects.

- **Endpoint:** POST `https://graph.facebook.com/v18.0/{phone_number_id}/media`

D. real-time Webhook Payload Handling

Processes inbound traffic and delivery states instantly. The backend application server listens on a validated public URL to process message updates:

```
// Process and routing layer for real-time payload updates
app.post('/whatsapp/webhook', (req, res) => {
  const payload = req.body;
  if (payload.object === 'whatsapp_business_account' && payload.entry) {
    const entryData = payload.entry[0].changes[0].value;

    // Check if the payload contains a status update
    if (entryData.statuses) {
      const statusUpdate = entryData.statuses[0];
      const messageId = statusUpdate.id;
      const deliveryStatus = statusUpdate.status; // sent, delivered, read
      // Execute database update logic here...
    }

    // Check if the payload contains an incoming message
    if (entryData.messages) {
      const inboundMsg = entryData.messages[0];
      const customerPhone = inboundMsg.from;
      const textContent = inboundMsg.text ? inboundMsg.text.body : "[Media Content]";
      // Execute WebSocket broadcast and DB write here...
    }
    return res.sendStatus(200);
  }
  res.sendStatus(404);
});
```

4. Broadcast Management & Pacing Controls

To avoid hitting limits or triggering spam flags, large campaigns cannot use standard loops. The platform utilizes a background task manager (like BullMQ or Celery) backed by Redis to control processing speed.

Pacing Notice: New numbers start out with message volume restrictions. Outbound tasks must be throttled to a maximum rate of 40 requests per second. The system should apply an exponential backoff retry strategy for transient errors, retrying failed attempts up to 3 times spaced out across 2^n seconds.

5. Operational Cost Matrix & Financial Projections

By shifting away from SaaS licensing models, the financial framework scales solely based on raw usage. This eliminates subscription markups and agent-seat caps entirely.

Expense Category	Custom Platform Setup	Third-Party Aggregators	Strategic Financial Benefit
Monthly Software Licensing	₹0 (Completely Free)	₹1,500 – ₹5,000+ / month	Eliminates recurring platform software costs permanently.
Agent License Seats	Unlimited (₹0)	Capped (Charges increase per agent)	Enables full team onboarding without additional seat costs.
Hosting Stack (VPS / Redis)	~₹1,500 – ₹3,000 / month	Included in Plan	Maintains full data control on isolated secure instances.
Outbound API Markups	₹0 (Direct Meta Rates Only)	₹0.10 – ₹0.20 per-message markup	Avoids intermediate processing fees on large campaigns.

Wholesale Meta Message Pricing (India Regional Matrix)

All message delivery costs are billed directly to your registered payment method by Meta, based on clear business categories:

- **Service Conversations (Support Actions): Free Tier.** Any inbound support session replied to within 24 hours carries zero cost.
- **Utility Conversations (Notifications/Alerts): ~₹0.115 per interaction.** Handles order receipts, confirmations, or delivery updates.
- **Authentication Conversations (Secure OTPs): ~₹0.115 per interaction.** Handles dynamic login codes.
- **Marketing Conversations (Promotional Broadcasts):** ~₹0.863 per interaction.** Handles discount campaigns or festive updates.

Estimated Bill Breakdown: For a typical monthly volume of 15,000 messages (comprising 7,000 Support chats, 5,000 Automated Utility updates, and 3,000 Promotional Marketing messages), the direct cost from Meta totals ₹3,164. Adding 18% GST (₹569) brings the final monthly operational cost to ₹3,733.

6. Phased Implementation Timeline

Building out this system follows a structured, step-by-step roadmap to ensure a secure and stable deployment:

1. **Phase 1: Meta Portal Configuration (Days 1–2):** Provisioning Developer apps, link verified company assets, onboarding clean numbers, and generating permanent admin authentication keys.

- 2. Phase 2: Data Architecture & Webhook Deployment (Days 3–5):** Instantiating PostgreSQL core models, coding the HTTPS webhook router, and managing end-to-end local data testing.
- 3. Phase 3: Multi-Agent Chat Panel (Days 6–9):** Building the front-end layout, linking live WebSockets, and setting up conversation routing for the team dashboard.
- 4. Phase 4: Bulk Distribution Setup & Analytics (Days 10–14):** Launching Redis worker threads, deploying template approval views, and completing end-to-end delivery performance tracking.